Section E - Performance Engineering Case Study

Scenario

QuickCart processes:

- 50 million clickstream events/day
- 5 million orders/day

Problems:

- SQL queries are slow
- Pipelines fail frequently
- Dashboard refresh takes 3 hours
- Storage cost is increasing

Tasks

Part 1 - Architecture Design

Design scalable architecture using:

- Database: Ingestion from database. Orders are transactional (OLTP)
- data lake: Store raw data in data lake (bronze layer) for cheap and scalable storage
- warehouse: used for analytics, holding cleaned, structured data optimized for dashboards and SQL querying.
- orchestration with airflow to create the DAG and scheduling the workflow
- transformation layer with dbt

Part 2 – Optimization

Explain:

1. Partitioning strategy: orders partitioned by order date and events partitioned by event date.
2. Indexing strategy: customer_id for customer lookups; order_time for orders filtering; customer_id (in orders table) and product_id (in order_items table) for joins.
3. Compression strategy: use parquet format to reduce storage and improve resources and analytics efficiency.
4. Query optimization: Avoid the use of SELECT *; optimize joins on indexed keys only; use of pre-aggregated tables.
5. Incremental loading: instead of reloading everything, load just new data. Use Watermarking (tracking the last processed timestamp) to only pull new records since the last successful run.
6. Columnar storage benefits using parquet over row-based storage as it improves compression, reduces scans and results in faster readings.

Part 3 – Reliability

Explain:

1. Retry handling : in order to reduce human intervention and improve reliability. Retries handle temporary failures: network issues, temporary resource unavailability, etc. Implement Exponential Backoff in Airflow, the system should wait 1 minute, then 2, then 4, before giving up.

2. Failure alerts: failure alerts are sent when there is a permanent failure or the failure needs humans to be fixed (code issue, data quality). When retires are exhausted for whatever reason.
3. Data quality validation: schema validation, duplicates, nulls and inconsistent data.
4. Monitoring strategy: Track Data Latency and Volume Anomalies. If you usually get 5M orders but suddenly see 500, trigger an alert for a potential upstream source failure
5. Idempotent design: Ensure that running the same pipeline twice results in the same state. Use UPSERT operations instead of blind APPEND. This prevents duplicate orders if a job fails halfway through and is restarted.